Assignment 1: The Turtlebot

Group 18

Leonardo Pontello
@leopont36

Marco Facco
@marcofacco2001

Giacomo Masiero
@giaco-mas

Introduction

The objective of the assignment 1 was to develop a navigation system for the Turtlebot3 within the 'LasLab'. Specifically, the goal was to identify the two specific AprilTag, and navigate to a calculated midpoint between them, and finally perform a table detection task. In this report we discuss the implementation of the nodes, and the solution that we have used.

Structure

We designed the system as a set of small nodes, each performing a clear and simple action. This makes the nodes easy to understand, modify, and reuse. We also used existing ROS 2 and Nav2 topics, services, and actions for communication between nodes whenever possible. This reduces the need for custom connections and makes the nodes ready to use in other ROS 2 projects.

LifeCycle Manager

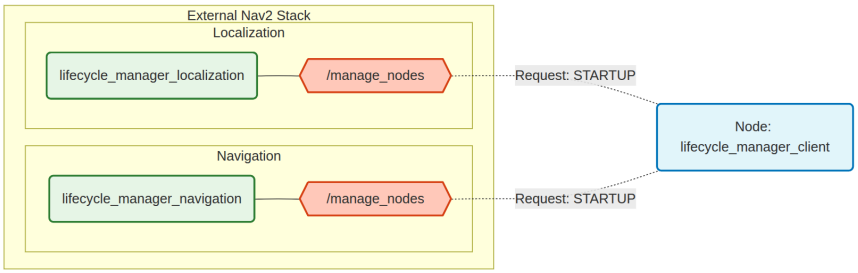

Figure 1: The LifeCycle Manager node

The first node is called 'LifeCycleManager', and is designed to start and coordinate the main module of Nav2 framework. In particular, it supervises the activation of the localization and navigation nodes by ensuring that they are launched in the correct order with a successfully transition to the 'active state'.
The startup procedure is executed in a dedicated thread, in a way that the nodes does not block the normal operation in Ros. After a short delay, the node attempts to activate the localization component first, and if succeeded proceed to start the navigation component. If the modules fails to respond, or encounters an issue, the node report the issue through log messages.

Initial Pose Setter

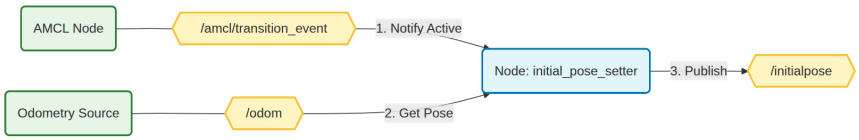

Figure 2: The PoseSetter Node

The 'InitialPoseSetter' node is designed to provide the robot initial pose to the navigation stack when the AMCL localization system becomes active. It subscribes to the AMCL lifecycle to detect when AMCL reaches the *active* state, and listen to the '/odom' topic to find the robot current pose. The node take the first available odometry message, converts it in the 'map' frame and publishes it to the '/initialpose'. This operation is repeated only once, to prevent repeated initialization.

Navigate to pose client

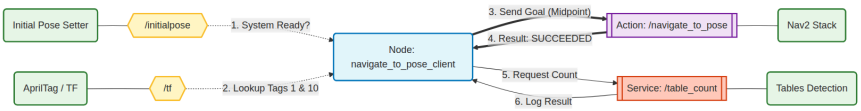

Figure 3: The NavigateToPoseClient node

The 'NavigateToPoseClient' acts as the central executive for the mission. It is responsible for determining the navigation goal and coordinating the robot's actions.

Instead of using hardcoded coordinates, the node dynamically calculates the destination by utilizing the `tf2_ros` transform library. It listens for the transforms of `tag36h11:1` and `tag36h11:10`. Once both tags are detected, the node computes the geometric midpoint between them.

The node implements a state machine using a timer callback:

1. It **waits for the `initial_pose_received` flag** (set as true when it gets published from InitialPoseSeter).
2. It continuously **queries the TF buffer** until transforms for both AprilTags are available.
3. Upon calculation, it **sends the goal** to the Nav2 `Maps_to_pose` action server.
4. Once the action succeeds (Goal Reached), it automatically **triggers the `table_count` service** to begin the detection phase.

Tables detection

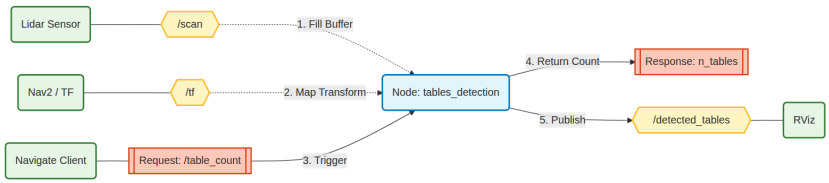

Figure 4: The TablesDetection Node

The `TablesDetection` node operates as a service server to identify round obstacles using 2D LiDAR data. To ensure robustness against noise and motion blur, the node buffers the last 15 laser scans and transforms all points into a fixed frame before processing.

The detection pipeline follows a three-stage approach:

1. **Clustering and Segmentation:** Laser points are grouped into clusters based on depth discontinuities.
2. **Geometric Fitting:** Each cluster is evaluated using a 3-point circumcenter algorithm. Candidates are validated based on convexity, radius constraints, and a Mean Squared Error (MSE) check to distinguish true circles from walls or corners.
3. **Temporal Voting:** Detections are spatially aggregated across the buffered scans. A voting mechanism filters out transient noise, confirming a table only if it is detected consistently across multiple frames (`MIN_VOTES`).

The confirmed counts are returned to the client, and a `PoseArray` is published to `detected_tables` (in `odom` ref frame) for visualization in Rviz.

Corridor Navigator

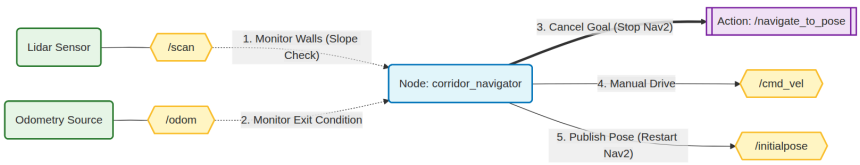

Figure 5: The CorridorNavigator Node

The 'CorridorNavigator' node enables the robot to automatically traverse narrow corridors, and act as action client for the 'NavigatoToPose' action. It operates by processing laser scan data to identify the structure and adjust the robot's velocity according to it. The node subscribe to the topic '/scan' to read the LaserScan data and to '/odom' to read odometry.

To detect a corridor, the node separates the laser points into left and right sets and, instead of performing a full linear regression, it directly computes the slope of each set. The two walls are considered parallel if the difference between the two slopes is below a predefined threshold.If both sides meet the criteria, the robot is considered to be inside the corridor. At this point, the node starts sending direct velocity commands through '/cmd_vel' to keep the robot centered, using the average wall slope to straighten the robot with respect to the corridor. When the robot leaves the corridor, the robot publish the initial pose on '/initial_pose' to reinitialize the Nav2 navigation.

Launch

In the launch file `group_18_assignment_1.launch.py`, all nodes were launched sequentially. The `LifecycleManager` node was started last to guarantee that every node was active before Nav2 Startup.

Test

To test the project, we launched the launch file and verified that the robot performed the expected behaviors both visually in Gazebo and RViZ, as well as by checking the log messages from the nodes. The video demonstrates the robot in action and shows the messages published by each node.

Links

- GitHub: https://github.com/marcofacco2001/group18_assignment_1
- Video: https://drive.google.com/file/d/1z7m9wzqKUqYG5wBUaFKPFHMx_5oekKLN/view?usp=sharing

Contributions

Leonardo Pontello(@leopont36) :

- Implemented the modules:
  - `LifecycleManager`
  - `InitialPoseSetter`
  - `CorridorNavigator`
- System integration (package.xml, CMakeLists.txt, launch files)
- Created the video showing the software in operation.

Marco Facco(@marcofacco2001)

- Implemented the modules:
  - `NavigateToPoseClient`
  - `TablesDetection`
- System integration (package.xml, CMakeLists.txt, launch files)
- Documentation.

Giacomo Masiero(@giaco-mas)

- Management of apriltag detection
- Documentation.